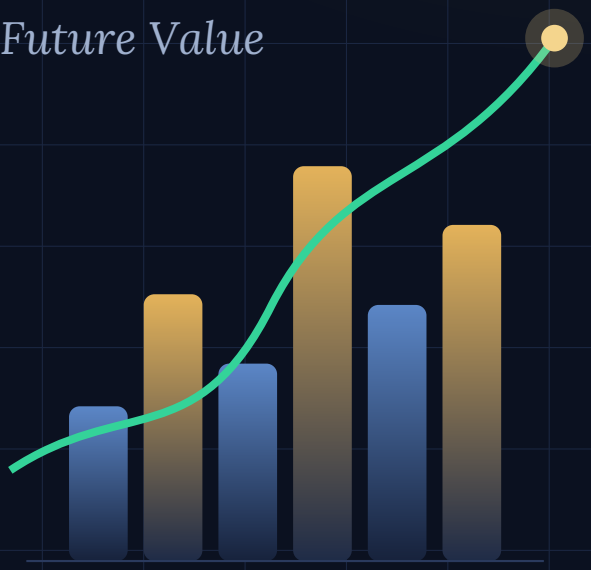
PROJECT DOCUMENTATION

Real Estate Investment Advisor

Predicting Property Profitability & Future Value

Methodology · Findings · Use cases · Operations

Classification and 5-year price forecasting for 250,000 Indian property listings, tracked with MLflow and served through a premium Streamlit application.



AUTHOR
Mohit Singh Rajput

VERSION
1.0

DATE
27 October 2026

REPOSITORY
github.com/13msrajput/Real-Estate-Investment-Advisor

LIVE APP
real-estate-investment-advisor-app.streamlit.app

PORTFOLIO
13msrajput.github.io

PYTHON · SCIKIT-LEARN · XGBOOST · MLFLOW · STREAMLIT · PLOTLY

Contents

1 Executive summary 4

2 Business context and use cases 6

3 Dataset and data quality 8

4 Solution architecture 10

5 Exploratory data analysis 12

6 Feature engineering and target design 15

7 Modelling methodology 18

8 Results 20

9 Robustness and honesty checks 23

10 Experiment tracking with MLflow 25

11 Application guide 26

12 Code base and engineering practices 33

13 Deployment and operations 36

14 Limitations, risks and roadmap 37

Appendix A Data dictionary 39

Appendix B Model feature dictionary 40

Appendix C Prediction service reference 41

Appendix D Glossary 43

Appendix E About the author 44

ABOUT THIS DOCUMENT

This document is the complete technical and functional reference for the **Real Estate Investment Advisor**. It follows the structure requested in the project brief (methodology, findings and use cases) and adds the operational detail needed to run, extend and audit the system. Every number was produced by the project's own code and is reproducible from the repository.

Audience	Data scientists, ML engineers, reviewers, and product owners evaluating the system.	**Conventions**	`Monospace` marks code, files and columns. Amounts are in Indian rupees; 1 lakh = ₹100,000 and 100 lakhs = 1 crore (Cr).

HOW TO READ THE CALLOUTS

Key insight (gold) marks takeaways, **Caution** (amber) limitations that affect how results should be read, and **Note** (blue) background information.

READING PATHS

Not every reader needs every section. Use these suggested paths.

IF YOU ARE ...	START WITH	THEN
A reviewer or product owner	1 Executive summary, 2 Business context	9 Robustness and honesty checks, 14 Limitations and roadmap
A data scientist	3 Dataset, 5 EDA, 6 Features and targets	7 Methodology, 8 Results, 9 Robustness
An ML or platform engineer	4 Architecture, 10 MLflow	12 Code base, 13 Deployment, Appendix C
A product or UX designer	11 Application guide	2 Business context (personas and journeys)
A new contributor	12 Code base, 13 Deployment	Appendix B (features), Appendix C (service API)

SECTION 1

Executive summary

01

A machine-learning application that tells an investor whether a property is a good investment, and what it could be worth in five years.

The **Real Estate Investment Advisor** answers two questions for a property in India: *is it a good investment?* (classification) and *what could its price be after five years?* (regression). It was built on **250,000 property listings** across 20 states and 42 cities and follows the project brief end to end: data preprocessing, exploratory analysis, model development, MLflow experiment tracking and a Streamlit application.

250,000	0.995	7.89 L	74%
PROPERTIES ANALYSED	**CLASSIFIER F1**	**FORECAST RMSE**	**LOWER ERROR**
20 states · 42 cities	XGBoost · 50,000 held out	₹ lakhs · 5-year price	than a fixed 8% growth rule

What was delivered

- **A validated cleaning pipeline** that surfaces the data's hidden quality problems, including **116,304 rows (46.5%) with a floor number above the building height**.
- **Answers to all 20 EDA questions** in the brief, supported by data-driven insights and charts.
- **30 model features and two transparent targets:** a four-factor *Good Investment* label and a feature-based five-year growth rate.
- **Three model families per task**, tuned XGBoost champions, and robustness checks (ablation, oracle benchmark, cross-validation).
- **MLflow tracking** with a model registry and `champion` aliases, **13 automated tests** and about 2,351 lines of commented code.
- **A premium Streamlit application** with live verdicts, forecasts with a 90% range, a property explorer, market insights and a model page, in light and dark themes.

Headline results

METRIC	VALUE	HOW TO READ IT
F1 score (classifier)	0.9951	Balances precision and recall on the minority 'good investment' class
ROC-AUC / accuracy	0.9999 / 99.69%	154 errors in 50,000 test properties
Precision of 'buy' signals	99.29%	Only 112 of 15,812 recommended properties were wrong
RMSE / MAE (regressor)	7.89 / 5.54 lakhs	R^2 0.9984, MAPE 1.64%
Versus fixed 8% growth rule	29.94 → 7.89 lakhs	74% lower RMSE
Versus the theoretical optimum	7.70 lakhs (oracle)	Champion is within 0.19 lakhs: the rest is random market noise
90% prediction range	0.966× to 1.033×	Held-out coverage 90.2%

HOW TO READ THESE NUMBERS

Read the results honestly. The dataset behaves like a generated snapshot: price is statistically independent of size, BHK, city and property type, and there is no price history.

The *Good Investment* label is **rule-based**, so the classifier's accuracy largely reflects recovering those rules. With every price column removed it falls to **71.9%** against a 68.5% majority-class baseline.

The five-year price is an **assumption-driven projection** (documented growth rates), not observed appreciation. The project reports these limits openly instead of hiding them.

Key findings

1. Hidden data-quality defects matter more than missing values: the file is perfectly complete yet 46.5% of floor numbers are impossible and the supplied price-per-sq-ft column is rounded to near uselessness.
2. Price and size are both uniformly distributed and uncorrelated (r = -0.003), so predictive power must come from engineered *ratios to a city benchmark*.
3. XGBoost wins both tasks. Against a fixed 8% growth rule it cuts forecast error by 74% and sits 0.19 lakhs from the noise floor.
4. Benchmarks (majority class, fixed rate, oracle, price-blind ablation) are what make the headline scores interpretable and honest.
5. The application exposes uncertainty (probability, 90% range) and reasoning (four-factor checklist) rather than a single unexplained number.

SECTION 2

Business context and use cases

02

Why the project exists, who it serves and how each requirement in the brief is satisfied.

Problem statement

Property is among the largest financial decisions most households make, yet many buyers rely on broker opinion or a bare price-per-square-foot comparison. The brief asks for a machine-learning application that assists potential investors by (1) classifying whether a property is a **Good Investment** and (2) predicting the estimated **property price after 5 years**, delivered through an interactive Streamlit application with experiment tracking in MLflow.

Business use cases

USE CASE IN THE BRIEF	HOW THE SOLUTION SUPPORTS IT	WHERE TO FIND IT
Empower investors with intelligent tools to assess long-term returns	Verdict with probability, five-year price with a 90% range, implied annual growth and a comparison with a fixed 8% benchmark	Advisor tab
Support buyers in choosing high-return properties in developing areas	Filter 250,000 properties by state, city, type, BHK, price, size and availability; location heatmaps and rankings	Explorer and Market insights tabs
Help real estate companies automate investment analysis for listings	`AdvisorService.predict()` turns one property dictionary into a full analysis; the same code path can serve batch scoring or an API	`src/predict.py`
Improve customer trust with data-backed predictions	Four-factor checklist explains every verdict; probabilities, prediction ranges, extrapolation warnings and an honest model-insights page	Advisor and Model insights tabs

Stakeholders and user journeys

PERSONA	GOAL	TYPICAL JOURNEY
Individual investor	Decide whether an asking price is justified	Enter details (or load an example) → read the verdict → check which factors failed → compare the price path with the 8% benchmark
Home buyer	Shortlist properties in a budget	Use the Explorer filters → sort by good-investment label and growth → download the shortlist as CSV
Platform / agency analyst	Add data-backed insight to listings	Call the prediction service for each listing; use city benchmarks and the reference tables
Data science team	Maintain and improve the models	Re-train with one command → compare runs in MLflow → promote a new `champion` version

Project evaluation criteria

CRITERION (BRIEF)	EVIDENCE IN THIS PROJECT
Data handling: imputation, encoding, outlier detection	Median / mode imputation logic, schema and range validation, one-hot encoding inside the pipeline, IQR outlier detection with winsorisation
Model evaluation: accuracy, F1, confusion matrix; RMSE, MAE, R^2	All reported, plus precision, recall, ROC-AUC and MAPE; confusion matrix, residuals and importance charts
Deployment: functional Streamlit UI for predictions and analytics	Four-tab application with input validation, live predictions, filters, market insights and model insights
Experimentation: version-controlled and logged with MLflow	Every run logged (parameters, metrics, figures, pipeline); registry with `champion` aliases; code in Git

Deliverables map

DELIVERABLE IN THE BRIEF	ARTEFACT
Cleaned and processed dataset (CSV or SQL)	data/processed/housing_processed.csv.gz plus city_summary.csv and year_trend.csv
Python scripts for EDA, model training and evaluation	real-estate-investment-advisor.ipynb and the src/ package (train.py, modeling.py, features.py, data_loader.py)
MLflow experiment logs and registered models	Created by src/train.py: experiments, runs and registry entries (see section 10)
Streamlit application (prediction and insights)	app.py with the design system in src/ui.py and the theme in .streamlit/config.toml
Project documentation (methodology, findings, use cases)	This document and README.md

SECTION 3

Dataset and data quality

03

What the data contains, what was wrong with it, and how every problem was handled.

Source and structure

The dataset (`india_housing_prices.csv`, 41 MB) holds **250,000 property listings described by 23 columns** across 20 states, 42 cities and 500 locality labels. Each row is one listing at a single point in time. The columns fall into five groups; the full dictionary is in Appendix A.

GROUP	COLUMNS
Location	`State, City, Locality`
Property	`Property_Type, BHK, Size_in_SqFt, Furnished_Status, Facing`
Price	`Price_in_Lakhs, Price_per_SqFt`
Building	`Year_Built, Age_of_Property, Floor_No, Total_Floors, Availability_Status`
Neighbourhood and ownership	`Nearby_Schools, Nearby_Hospitals, Public_Transport_Accessibility, Parking_Space, Security, Amenities, Owner_Type`

Data-quality audit

The file contains no missing values and no duplicates, which is unusual for real-world data. A deeper audit of the *values* found the problems below.

CHECK	FINDING	ACTION TAKEN
Missing values	None in any of the 23 columns	Median / mode imputation logic retained for future data
Duplicate listings	None (ignoring the ID column)	Duplicates would be dropped
Floor above building height	**116,304 rows (46.5%)** (e.g. floor 22 of a 1-storey building)	Capped at Total_Floors; flag column Floor_Data_Corrected added
Price per sq ft precision	Stored in lakhs with two decimals (0.05, 0.10 …), destroying almost all information	Rebuilt in rupees from price and size: Price_per_SqFt_INR
Age consistency	Age_of_Property equals 2025 − Year_Built in every row	Recomputed anyway to guarantee consistency
Locality labels	500 generic labels, each appearing in all 42 cities	Excluded from model features (not a geographic identifier)
Distribution shape	Price (10–500 lakhs) and size (500–5,000 sq ft) are uniform; size–price r = -0.003	Documented; engineered ratios to city benchmarks used instead

CAUTION

Prices in this dataset are statistically independent of size, BHK, city and property type. That is unrealistic for a real market, and it shapes every modelling decision in this document. The project states the limitation explicitly rather than presenting spurious real-world conclusions.

Cleaning pipeline

`clean_data()` works on a copy of the raw frame and returns both the cleaned data and an **audit report** recording what each step changed.

#	STEP	ROWS / CELLS AFFECTED
1	Strip whitespace; convert empty strings to nulls	all text columns
2	Drop duplicate listings (ignoring ID)	0
3	Drop rows with unusable price, size, BHK or year	0
4	Impute remaining gaps (median for numbers, mode for categories)	0 cells
5	Enforce business rules (positive price, size, BHK, floors; year built ≤ 2025)	0
6	Recompute Age_of_Property from Year_Built	0
7	Cap impossible floors and add Floor_Data_Corrected	**116,304**
8	Rebuild price per sq ft in rupees	all rows
9	Standardise Yes/No columns (Parking_Space, Security)	all rows

Result: 250,000 clean rows. Capped rows are kept rather than deleted, because removing almost half of the data would be far more damaging than a capped floor value.

> **NOTE**
> Capping floors at the building height creates a correlation of **+0.58** between Floor_No and Total_Floors in the cleaned data. In the raw file it is +0.001. This is a documented side effect of the fix, not a property of the market.

Outlier handling

The 1.5 × IQR rule finds no outliers in price or size (both are bounded), but **19,723 properties (7.9%) have a price per sq ft above the fence of ₹32,764**, reaching ₹99,182. These are small flats with large price tags, created by independent price and size values. They are **winsorised at the 1st and 99th percentiles** (₹498 to ₹65,823) in the model feature `Price_per_SqFt_Capped`, while the raw value is preserved for the investment-label rules. Tree models are insensitive to the remaining skew.

SECTION 4

Solution architecture

04

How data flows from the raw file to a verdict on the screen, and how the code base is organised.

Training pipeline

Training is a single command (`python -m src.train`). Each stage is a small, testable function; the notebook, the training script and the app all import the same modules.

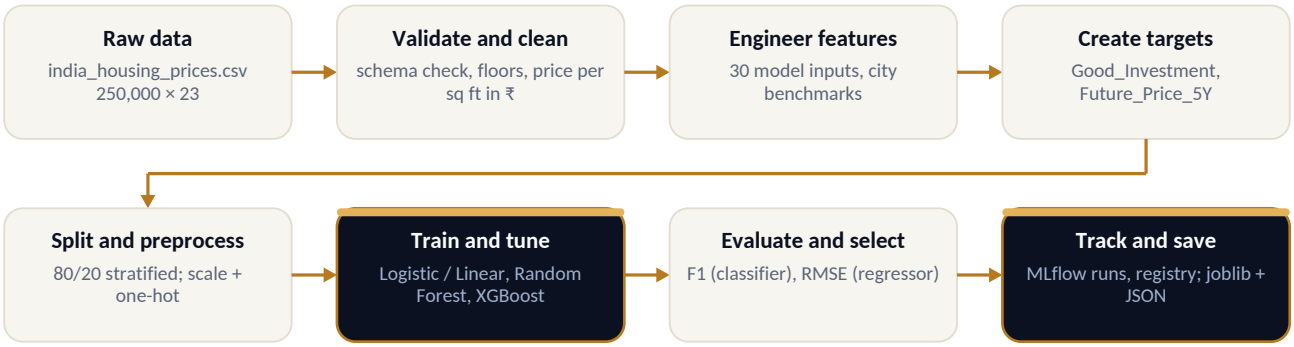


Figure 1. *Training pipeline. Gold-edged stages produce the champion models and their tracked artefacts.*

Inference flow

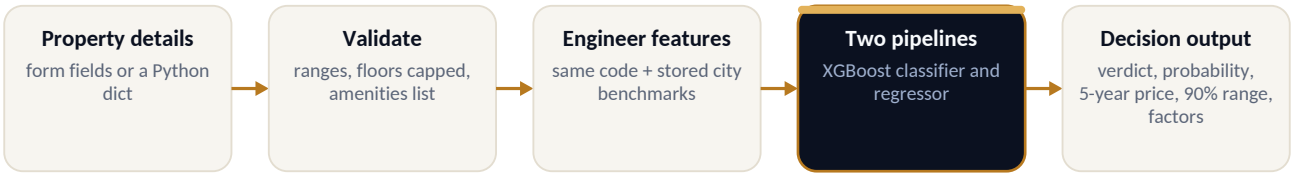


Figure 2. *Inference flow used by the Streamlit app. Feature engineering at inference is the same function used in training.*

Module map

FILE	RESPONSIBILITY	LINES
`src/config.py`	Paths, seeds, growth assumptions, infrastructure weights, MLflow names	88
`src/data_loader.py`	Loading, schema validation, cleaning audit trail, IQR outliers, winsorising	173
`src/features.py`	Row-wise and market-relative features, benchmark tables, growth rate, targets	219
`src/modeling.py`	Preprocessing, model factories, tuning, evaluation, importance, persistence	244
`src/train.py`	Command-line training with MLflow tracking, registry and figures	383
`src/predict.py`	`AdvisorService`: input validation and inference for the app and scripts	123
`src/ui.py`	Design system: palettes, CSS, HTML components, chart styling	402
`app.py`	Streamlit application (four tabs)	560
`tests/`	13 unit tests (pipeline and prediction service)	134

Artefacts

ARTEFACT	PURPOSE	SIZE
`models/classifier_pipeline.joblib`	Preprocessing + XGBoost classifier in one object	0.39 MB
`models/regressor_pipeline.joblib`	Preprocessing + XGBoost regressor in one object	0.31 MB
`models/reference_tables.json`	City medians, demand index, thresholds used to engineer features for new properties	7.7 KB
`models/model_metadata.json`	Metrics, importances, interval, assumptions shown in the app	6.5 KB
`data/processed/housing_processed.csv.gz`	Cleaned, engineered dataset with both targets	27.4 MB
`data/processed/city_summary.csv, year_trend.csv`	Pre-aggregated tables for dashboard charts	4.2 KB

Design principles

- **One source of truth.** The notebook, training script and app share the same modules, so a number cannot differ between them.
- **Preprocessing travels with the model.** Encoding and scaling live inside the saved `Pipeline`, so serving can never apply a different transform from training.
- **Validate at the boundaries.** Files, user inputs and model artefacts are checked where they enter the system, with messages a user can act on.
- **Centralised assumptions.** Seeds, growth rates, weights and paths live in `src/config.py`; changing an assumption is a one-line edit.
- **Reproducible by construction.** Fixed seeds, a stratified split, exact library pins and an audit trail from raw data to metrics.

SECTION 5

Exploratory data analysis

05

Answers to the 20 questions in the brief, grouped as price and size, location, feature relationships, and investment, amenities and ownership.

The analysis uses the cleaned data. Four figures summarise the evidence; the table that follows answers each of the 20 questions with the numbers behind it. All charts with executed code are in the notebook (28 charts).

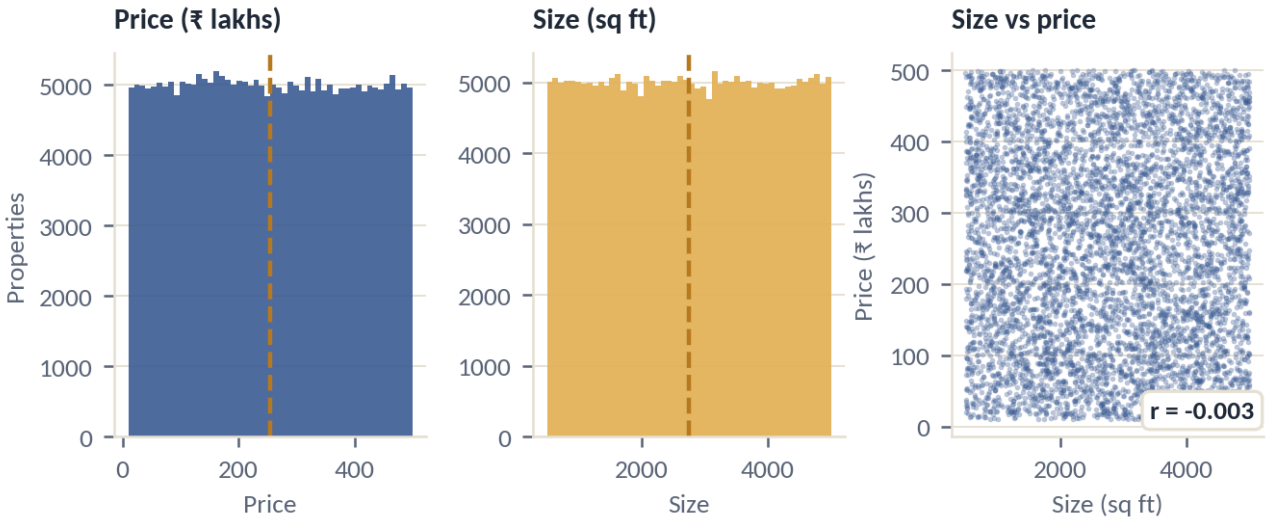


Figure 3. *Distributions of price and size (dashed lines mark the means) and size versus price. Both variables are uniform and the correlation is -0.003 (questions 1, 2, 4).*



Figure 4. *Average price by category. Every axis starts at zero and uses the same scale: furnishing, facing, owner type, parking, transport access and property type make no visible difference (questions 3, 14–18, 20).*

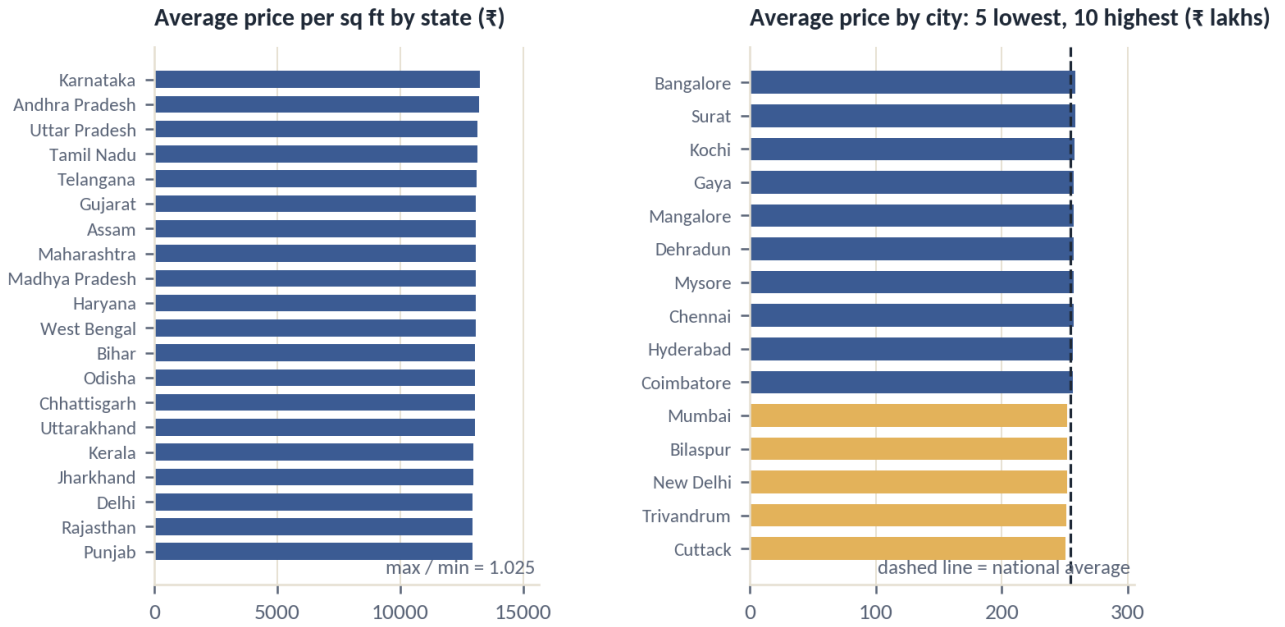


Figure 5. Average price per sq ft by state (max/min = 1.025) and average price by city (questions 6 and 7). Geography barely moves price.

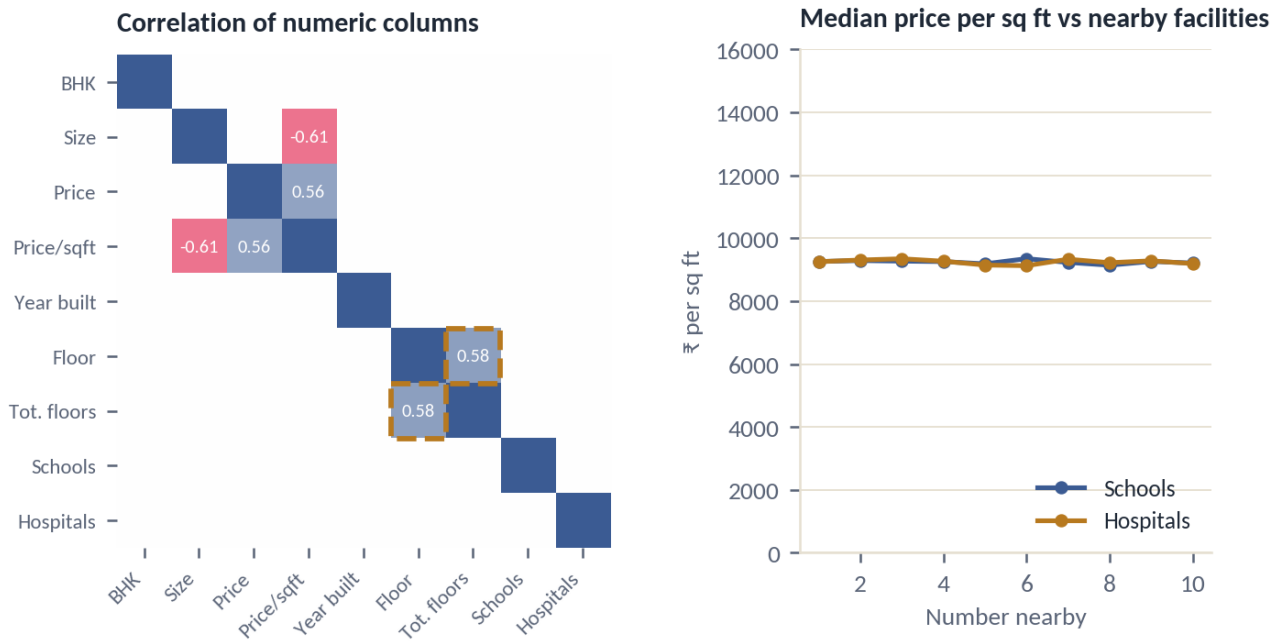


Figure 6. Left: correlations between numeric columns. The dashed outline marks Floor × Total floors, whose +0.58 is created by the floor-capping step. Right: nearby schools and hospitals have no relationship with price (questions 11–13).

KEY INSIGHT

What the EDA means for modelling. Because raw columns carry almost no cross-feature signal, the project engineers *relative* features: price and price per sq ft as ratios to the city (or city-and-type) median, an infrastructure score, and a locality-relative school score. These are the quantities an investor actually reasons about.

Infrastructure, parking and amenities are kept in the investment rules even though they do not move historical price here, because a buyer values them independently of how this snapshot was generated.

Findings for the 20 questions

#	QUESTION	FINDING
1	**Distribution of prices**	Uniform from 10 to 500 lakhs; each fifth of the range holds 19.9%–20.3% of properties
2	**Distribution of sizes**	Uniform from 500 to 5,000 sq ft; each fifth holds 20.0%–20.0%
3	**Price per sq ft by property type**	Median differs by only 1.2% across Apartment, House and Villa
4	**Relationship between size and price**	None: r = -0.003. Price per sq ft is strongly negative with size (-0.61) purely through the formula
5	**Outliers in price per sq ft or size**	Size and price: none. Price per sq ft: 19,723 outliers (7.9%), winsorised
6	**Average price per sq ft by state**	Ranking exists but spread is only 2.5% of the mean
7	**Average price by city**	Ranges 250.8–258.5 lakhs, a spread of 3.0%
8	**Median age by locality**	Locality medians span 16–21 years; every locality contains ages 2 to 35
9	**BHK distribution across cities**	Each BHK is 18.6%–22.0% of listings in every city; size does not scale with BHK
10	**Price trend, five most expensive localities**	Top five sit only +14.5 to +20.8 lakhs above the mean; directions are mixed (noise). No dates exist, so no true trend is measurable
11	**Correlation of numeric features**	Only structural pairs correlate (age/year, price per sq ft vs price and size, floor capping); everything else is within ±0.006
12	**Schools vs price per sq ft**	r = -0.000; flat across 0–10 schools
13	**Hospitals vs price per sq ft**	r = -0.000; flat across 0–10 hospitals
14	**Price by furnished status**	Gap of 0.3% between the three groups
15	**Price per sq ft by facing direction**	Gap of 1.0% between the four directions
16	**Properties per owner type**	Balanced: Broker 33.4%, Owner 33.3%, Builder 33.3%
17	**Properties per availability status**	Split 50.0% / 50.0% between Under_Construction and Ready_to_Move
18	**Does parking affect price?**	No: +0.12% difference
19	**Do amenities affect price per sq ft?**	No: each amenity shifts the median by Gym +0.5%, Pool +0.1%, Clubhouse -0.1%, Garden +0.1%, Playground +0.3%
20	**Transport access vs price and investment potential**	Price: flat. Good-investment share rises 19.2% → 44.3% Low → High, by construction of the rules

SECTION 6

Feature engineering and target design

06

From 23 raw columns to 30 model inputs and two documented targets.

Engineered features

FEATURE	DEFINITION	WHY IT MATTERS
`Price_per_SqFt_INR`	Price_in_Lakhs × 100,000 / Size_in_SqFt	Standard value metric requested in the brief
`PPS_vs_City_Median`	Price per sq ft ÷ median of the property's city	Below 1 means cheaper than the local market
`Price_vs_City_Median`	Price ÷ median price of the same city and property type	Affordability versus comparable homes
`School_Density_Score`	Nearby schools ÷ average schools in the city	Locality-relative education access (brief)
`Transport_Score`	Low = 1, Medium = 2, High = 3	Ordinal connectivity
`Amenity_Count`, `Has_<amenity>`	Parsed from the comma-separated Amenities text	Lifestyle features in numeric form
`Infra_Score`	Weighted 0–100 score (see below)	Single infrastructure measure requested in the brief
`Floor_Ratio`	Floor_No ÷ Total_Floors	Position in the building (0 ground, 1 top)
`Size_per_BHK`	Size_in_SqFt ÷ BHK	Spaciousness proxy
`City_Demand_Index`	Rank of the city's median price per sq ft, scaled 0–1	Market demand level
`Is_Under_Construction`	1 when Availability_Status is Under_Construction	Delivery risk and completion upside

INFRASTRUCTURE SCORE WEIGHTS

COMPONENT	WEIGHT	NORMALISATION
Nearby schools	25%	count ÷ 10
Nearby hospitals	25%	count ÷ 10
Public transport	25%	(score − 1) ÷ 2
Parking	10%	1 if available
Amenities	10%	count ÷ 5
Security	5%	1 if available

Classification target: Good Investment

The brief asks for a binary label "based on domain rules". A property earns **one point for each of four factors** and is labelled good when it earns **at least three**. The rules are transparent, auditable and mirror the brief's examples (price versus median, price per sq ft, a multi-factor score).

FACTOR	RULE	BRIEF REFERENCE
Value	Price per sq ft ≤ the city median	Price per sq ft
Affordability	Price ≤ median price for the same city and property type	Price vs median
Infrastructure	Infra_Score ≥ 57.5 (top 40% of properties)	Multi-factor score
Growth	Expected annual growth ≥ 6.94% (median)	Appreciation rate > threshold

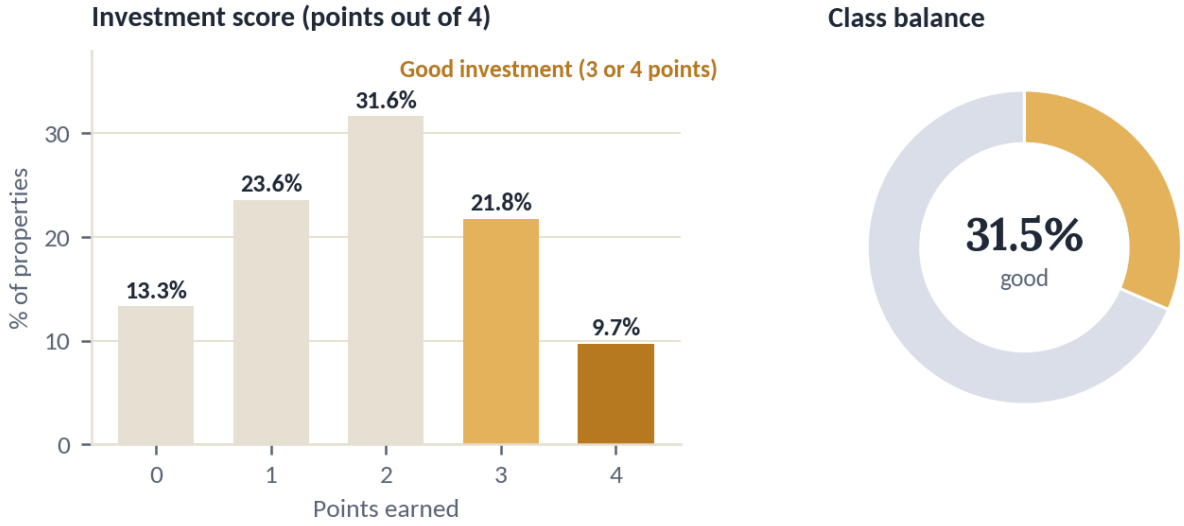


Figure 7. Left: how many points each property earns (gold bars qualify as good). Right: the resulting class balance, 31.5% good, handled with class weights rather than resampling.

Regression target: price after five years

The dataset has no price history, so true appreciation cannot be learned. Of the three options in the brief (fixed rate, location-based, feature-based) the project uses the **feature-based** approach and keeps the fixed 8% rate as a benchmark the model must beat:

Future_Price_5Y = Price × (1 + r) ^ 5	r = 0.045 + 0.030 × CityDemand + 0.020 × Infra/100 + 0.008 × UnderConstruction − 0.0005 × max(Age − 10, 0) + noise

Reproducible Gaussian market noise (standard deviation 0.4% a year) is added and growth is clipped to 3%–12%. The average property grows 6.95% a year, so it gains about 40% over five years.

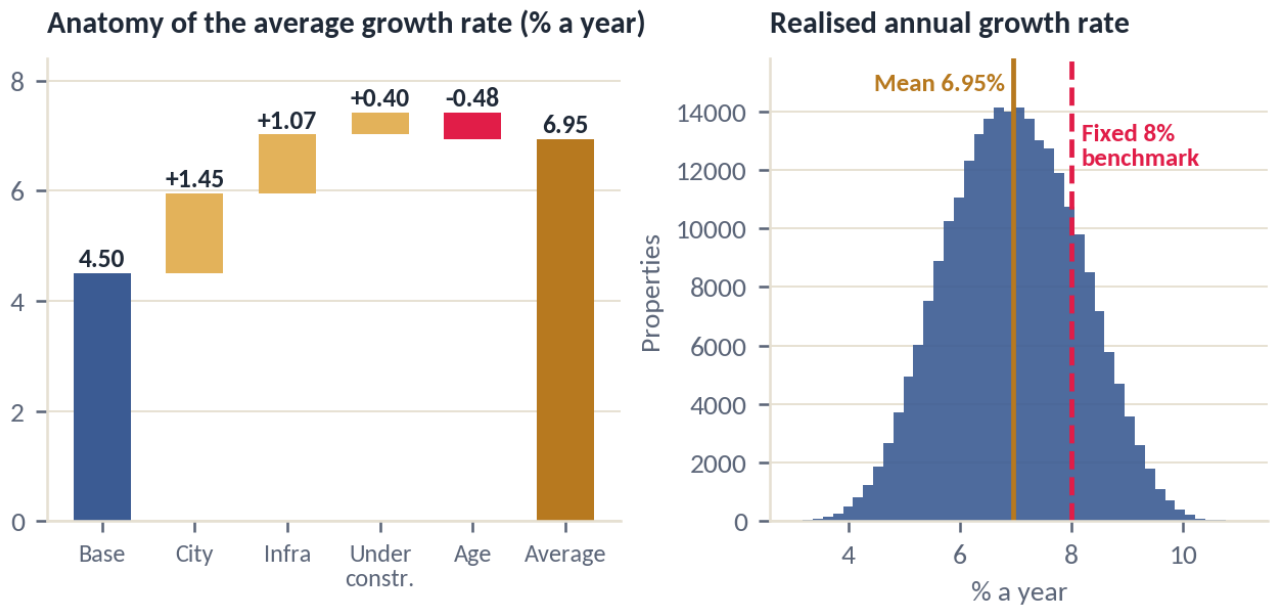


Figure 8. Left: contribution of each assumption to the average growth rate. Right: realised growth rates against the fixed 8% benchmark.

CAUTION

Both targets are defined from the features. The models learn the investment philosophy encoded in these rules and assumptions; they do not discover market behaviour hidden in the data. Section 9 quantifies exactly how much of the performance is rule recovery.

City benchmarks (medians, demand index, thresholds) are computed once on the full dataset and stored as reference tables, so they act as fixed domain constants at inference time rather than learned parameters.

SECTION 7

Modelling methodology

07

Preprocessing, data splitting, candidate algorithms, hyper-parameter tuning and evaluation metrics.

Preprocessing and split

- **25 numeric features** are standardised with `StandardScaler`; **5 categorical features** (City, Property_Type, Furnished_Status, Facing, Owner_Type) are one-hot encoded with `handle_unknown="ignore"`, giving **80 model columns**. An unseen city produces an all-zero row instead of an error.
- State is dropped because every city belongs to one state; Locality is dropped because its labels are generic.
- The encoder and scaler live **inside the scikit-learn Pipeline**, so they are fitted on training data only and saved with the model.
- An **80/20 split** (200,000 training and 50,000 test properties) is stratified on the classification label with seed 42; both tasks share the same split.

Candidate algorithms

TASK	MODEL	RATIONALE	KEY SETTINGS
Classification	**Logistic Regression**	Interpretable linear baseline	max_iter 1000, class_weight balanced
	Random Forest	Bagged trees capture threshold rules	120 trees, depth 14, min leaf 5, 60% rows per tree, balanced_subsample
	XGBoost (tuned)	Gradient boosting, strong on tabular rules	400 trees, depth 6, lr 0.15, subsample 0.85, colsample 1.0, min child weight 10, scale_pos_weight 2.18
Regression	**Linear Regression**	Baseline; future price is proportional to price	ordinary least squares
	Random Forest	Non-linear baseline	100 trees, depth 14, min leaf 5, 60% rows per tree
	XGBoost (tuned)	Learns the growth function on top of price	200 trees, depth 6, lr 0.05, subsample 0.7, colsample 0.8, min child weight 1

Hyper-parameter tuning

XGBoost is tuned with a **randomised search**: 6 random candidates from the grid below, each scored with 3-fold cross-validation on a reproducible 40,000-row subsample (F1 for the classifier, negative RMSE for the regressor). Only the winning parameters are used for the final fit on all 200,000 training rows. If tuning fails the code falls back to sensible defaults instead of stopping.

PARAMETER	VALUES SEARCHED	CLASSIFIER WINNER	REGRESSOR WINNER
`n_estimators`	200, 300, 400	400	200
`max_depth`	4, 6, 8	6	6
`learning_rate`	0.05, 0.10, 0.15	0.15	0.05
`subsample`	0.70, 0.85, 1.00	0.85	0.7
`colsample_bytree`	0.6, 0.8, 1.0	1.0	0.8
`min_child_weight`	1, 5, 10	10	1

Evaluation metrics

METRIC	MEANING	ROLE
Accuracy	Share of correct predictions	Reported
Precision	Of properties flagged good, the share that truly are	Reported (cost of a false 'buy')
Recall	Of truly good properties, the share found	Reported
F1	Harmonic mean of precision and recall	**Selects the classifier champion**
ROC-AUC	Ranking quality across all thresholds	Reported
RMSE	Root mean squared error in ₹ lakhs; penalises large misses	**Selects the regressor champion**
MAE / MAPE	Average absolute error (lakhs) / as a percentage of the true price	Reported
R^2	Share of variance explained	Reported (interpret with care, see section 8)

NOTE

The classes are imbalanced (31.5% positive). Logistic Regression and Random Forest use balanced class weights; XGBoost uses `scale_pos_weight` = 2.18 (negatives ÷ positives in the training set).

SECTION 8

Results

08

XGBoost wins both tasks. This section reports every metric on the 50,000 held-out test properties.

Classification: Good Investment

MODEL	ACCURACY	PRECISION	RECALL	F1	ROC-AUC
Logistic Regression	0.8814	0.7663	0.8967	0.8264	0.9585
Random Forest	0.9690	0.9157	0.9929	0.9528	0.9980
XGBoost	**0.9969**	**0.9929**	**0.9973**	**0.9951**	**0.9999**

XGBoost is the champion with F1 **0.9951**. Logistic Regression reaches only 0.8264 because it cannot express the 'is this ratio below one?' threshold rules directly; Random Forest (0.9528) approximates them but with limited depth.

Regression: price after five years

MODEL	RMSE (LAKHS)	MAE (LAKHS)	R^2	MAPE
Linear Regression	12.723	9.641	0.9959	6.02%
Random Forest	8.296	5.810	0.9983	1.67%
XGBoost	**7.891**	**5.537**	**0.9984**	**1.64%**
Fixed 8% growth rule (benchmark)	29.942	21.627	0.9775	6.29%
Oracle: knows the noise-free growth formula	7.701	5.363	0.9985	1.50%

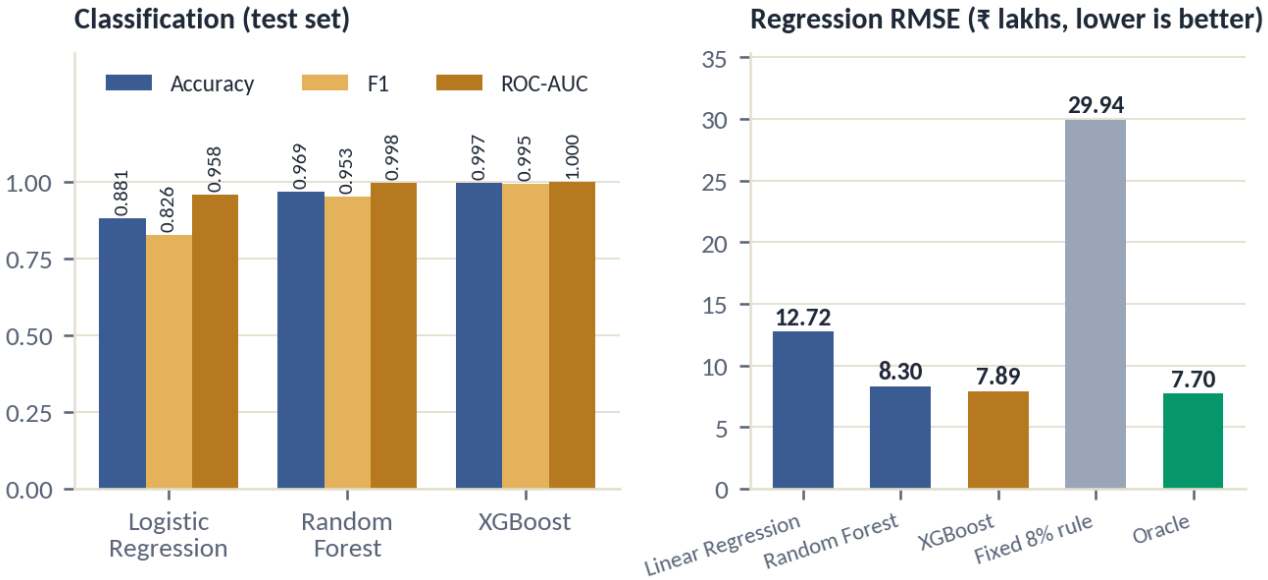


Figure 9. *Test-set comparison of all candidate models. Right: forecast RMSE including the fixed 8% rule and the oracle (the best any model could achieve given the random noise term).*

KEY INSIGHT

R^2 values near 0.998 should not be over-interpreted: current price alone explains almost all the variance of any future price, so even the fixed 8% rule reaches R^2 0.978. The honest measure of value added is the RMSE reduction (29.9 → 7.9 lakhs) and the distance to the oracle (7.70 lakhs).

Errors and confidence

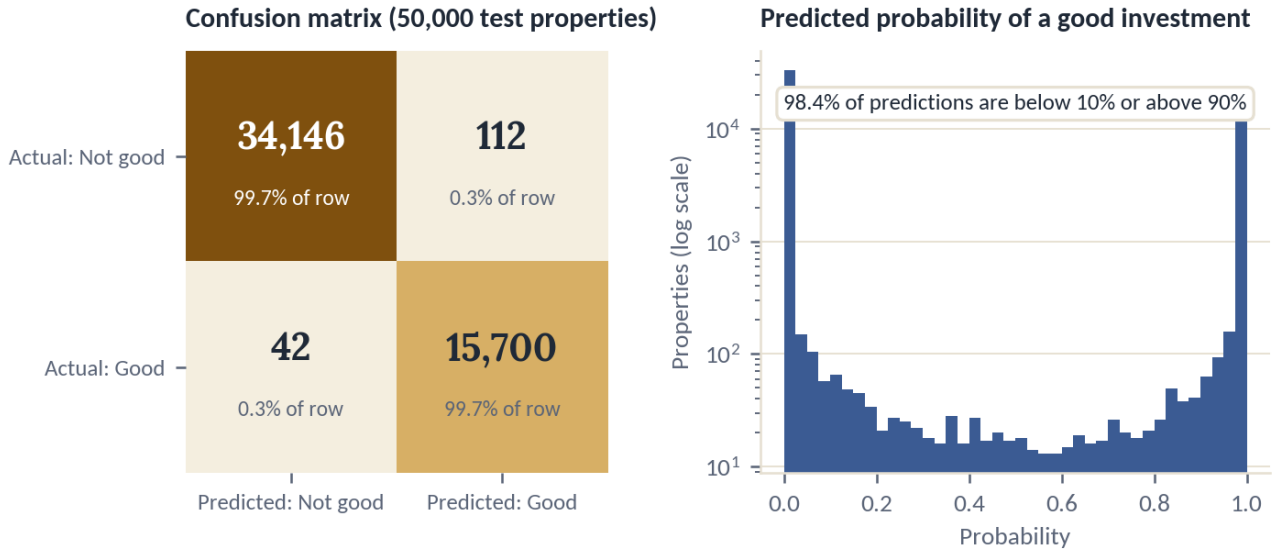


Figure 10. Left: confusion matrix of the champion classifier. Right: predicted probabilities; 98.4% are below 10% or above 90%, as expected for a rule-based label.

- Of 50,000 test properties the champion makes **154 errors**: 112 false alarms and 42 missed opportunities.
- Only **0.71%** of the properties it recommends are wrong, which matters because a false 'buy' signal costs more than a missed opportunity.
- The app displays probabilities of exactly 0 or 1 as `>99%` or `<1%`, because a model is never literally certain.

What drives the predictions

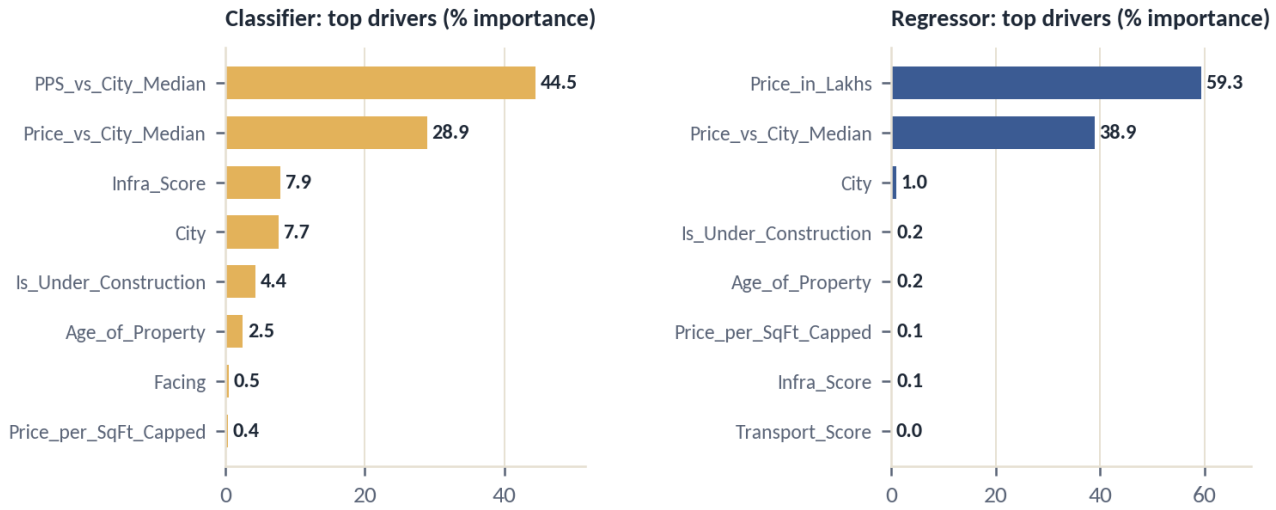


Figure 11. Feature importance (gain) of the champions, with one-hot columns summed back to their parent feature.

- **Classifier:** `PPS_vs_City_Median` (44.5%), `Price_vs_City_Median` (28.9%), `Infra_Score` (7.9%), `City` (7.7%). These are exactly the quantities used to define the label, which is the signature of *rule recovery*.
- **Regressor:** `Price_in_Lakhs` (59.3%), `Price_vs_City_Median` (38.9%). Price_vs_City_Median is simply price divided by a city benchmark. City, infrastructure and age matter only through the growth rate, so their individual importance is small.

Regression diagnostics and prediction range

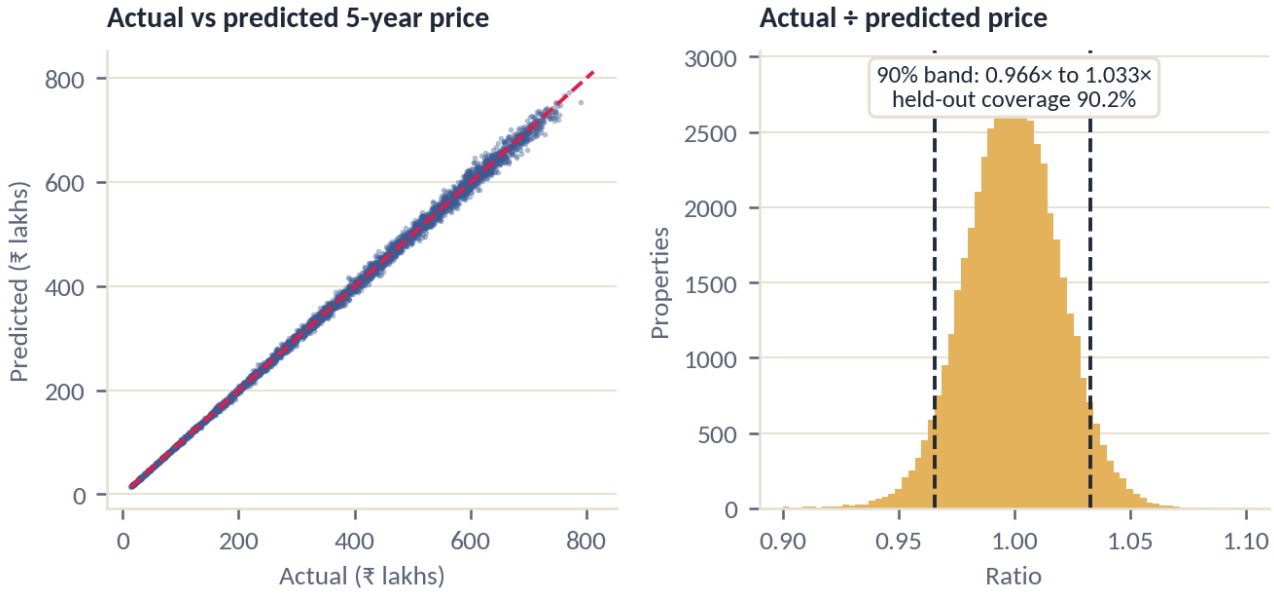


Figure 12. *Left: predictions hug the diagonal across the whole price range. Right: actual ÷ predicted price; the dashed lines are the 5th and 95th percentiles (0.966× and 1.033×).*

The app multiplies each point forecast by the two ratios to show a **90% prediction range**. The band is estimated on one half of the test set and checked on the other half, giving an empirical coverage of **90.2%**, so the interval is honestly calibrated rather than tuned to look good.

SECTION 9

Robustness and honesty checks

09

High scores are only meaningful if they survive scrutiny. These checks show what the models do and do not prove.

Leakage ablation: removing every price feature

Two of the four label factors compare price with a city benchmark. Removing all price-based columns (price, price per sq ft and their ratios) leaves only the infrastructure and growth information, so the comparison shows how much of the accuracy is rule recovery.

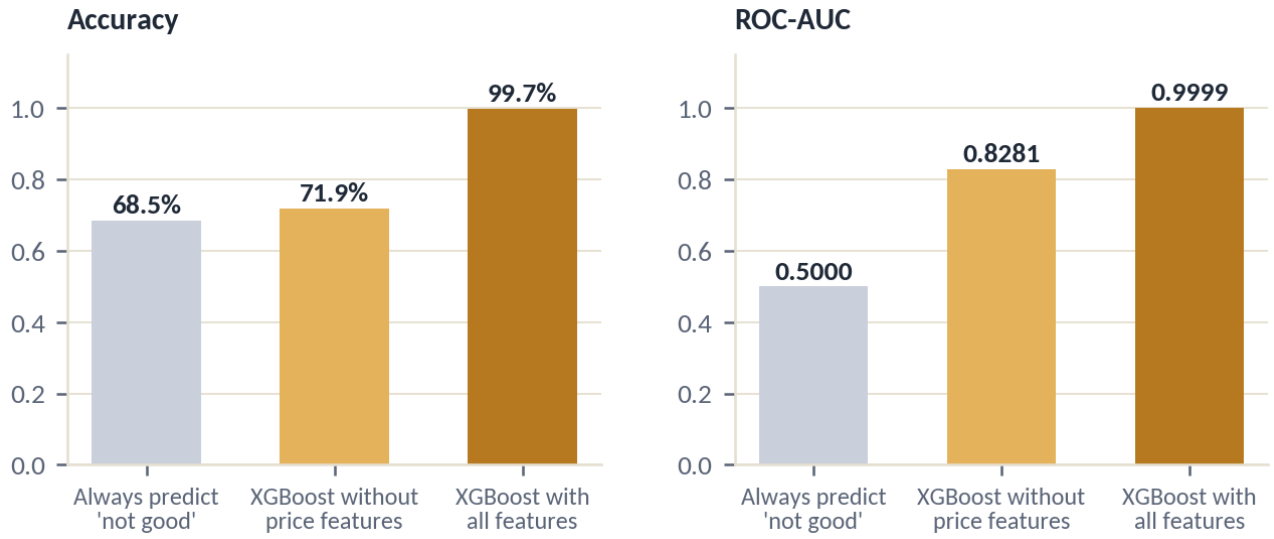


Figure 13. *Same XGBoost configuration with and without price features, compared with always predicting 'not good'.*

- With all features: accuracy **99.69%**, ROC-AUC 0.9999. Without price features: **71.9%**, ROC-AUC 0.828.
- The majority-class baseline already scores 68.5%, so the price-blind model adds only 3.4 accuracy points, although its ROC-AUC shows it still ranks properties sensibly.
- **Conclusion:** the jump from about 72% to 99.7% comes from price features that define the value and affordability factors. The classifier is a faithful implementation of the investment rules, which the brief asks for, but it is not evidence of predictive power on real market returns.

Benchmarks for the regressor

- A fixed 8% rule gives RMSE **29.94 lakhs**; the champion **7.89**; the oracle **7.70**.
- The champion closes **99%** of the gap between the naive rule and the theoretical optimum, so little remains to be gained without changing the problem.
- The remaining error is the random market-noise term (standard deviation 0.4% a year), which is unpredictable by construction and is why a prediction range is shown.

Cross-validation stability

FOLD	CLASSIFIER F1	REGRESSOR RMSE (LAKHS)
1	0.9921	8.167
2	0.9885	8.187
3	0.9870	8.154
4	0.9895	8.094
5	0.9868	7.866
Mean ± std	**0.9888 ± 0.0019**	**8.094 ± 0.118**

Five-fold cross-validation on a 40,000-row sample of the training data shows a very small spread, so the test scores are not the product of a lucky split. Absolute values are slightly worse than the full-data test scores because only a fifth of the training data is used.

Overfitting check

MODEL	TRAIN F1	TEST F1	GAP
Logistic Regression	0.8302	0.8264	+0.0038
Random Forest	0.9606	0.9528	+0.0078
XGBoost	0.9999	0.9951	+0.0048

The largest train–test gap is 0.008 F1, so there is no meaningful overfitting.

What the results do and do not show

THE RESULTS DO SHOW	THE RESULTS DO NOT SHOW
The pipeline cleans, engineers and models the data correctly and reproducibly	That price in this dataset is explained by size, location or amenities (it is not)
XGBoost recovers the four-factor investment rules almost perfectly	That the classifier predicts real-world investment returns
The forecast beats a fixed 8% rule by 74% and sits near the noise floor	That growth rates of 3–12% a year reflect real market appreciation
The 90% range is calibrated on held-out data (90.2% coverage)	That city-level differences in growth are real: the demand index stretches a 5% spread in median price per sq ft

SECTION 10

Experiment tracking with MLflow

10

Every run is logged and the champions are registered, so results are reproducible and models can be promoted or rolled back.

What is tracked

ITEM	DETAIL
Backend	Local SQLite database (mlflow.db) with artefacts in mlartifacts/: the SQLite backend is required for the model registry
Parameters, metrics, tags	Every hyper-parameter; accuracy, precision, recall, F1, ROC-AUC (classification); RMSE, MAE, R^2, MAPE (regression); task and stage tags
Artefacts	Fitted scikit-learn pipeline with input/output signature; for each champion a report run with confusion matrix, ROC, actual-vs-predicted, residual and importance figures

Experiments and registry

EXPERIMENT / REGISTRY ENTRY	RUNS	SELECTION METRIC
`RealEstate_GoodInvestment_Classification`	4 (3 models + champion report)	F1 (highest)
`RealEstate_FuturePrice_Regression`	4 (3 models + champion report)	RMSE (lowest)
`RealEstate_GoodInvestment_Classifier`	version 1, alias champion	XGBoost
`RealEstate_FuturePrice_Regressor`	version 1, alias champion	XGBoost

Re-training adds a new registry version; moving the `champion` alias promotes or rolls back a model without touching application code. Models are logged with the `cloudpickle` format because MLflow 3's default `skops` format refuses to serialise tree ensembles without explicit trust.

Working with the store

```
# Launch the tracking UI from the project root, then open http://localhost:5000
mlflow ui --backend-store-uri sqlite:///mlflow.db

# Load the registered champion in Python
import mlflow
mlflow.set_tracking_uri("sqlite:///mlflow.db")
model = mlflow.sklearn.load_model("models:/RealEstate_GoodInvestment_Classifier@champion")
```

NOTE

The MLflow store is git-ignored (its artefact paths are machine-specific). Run `python -m src.train` once to create it.

SECTION 11

Application guide

11

A tour of the Streamlit application, its design system and the safeguards behind the interface.

Overview of the four tabs

TAB	PURPOSE	KEY ELEMENTS
Advisor	Analyse one property	One-click examples, live form, verdict ring, KPI cards, price path with 90% range, four-factor checklist, extrapolation warnings
Explorer	Shortlist from 250,000 properties	Filters (state, city, type, BHK, price, size, status, good only), KPI cards, table with progress bars, CSV download, price histogram
Market insights	Understand locations	State → city treemap, state × type heatmap, top cities, infrastructure-vs-growth bubbles, trend charts
Model insights	Audit the models	Champion metrics, confusion matrix, error vs benchmark, importance, all candidates, honest interpretation, model card

Advisor tab

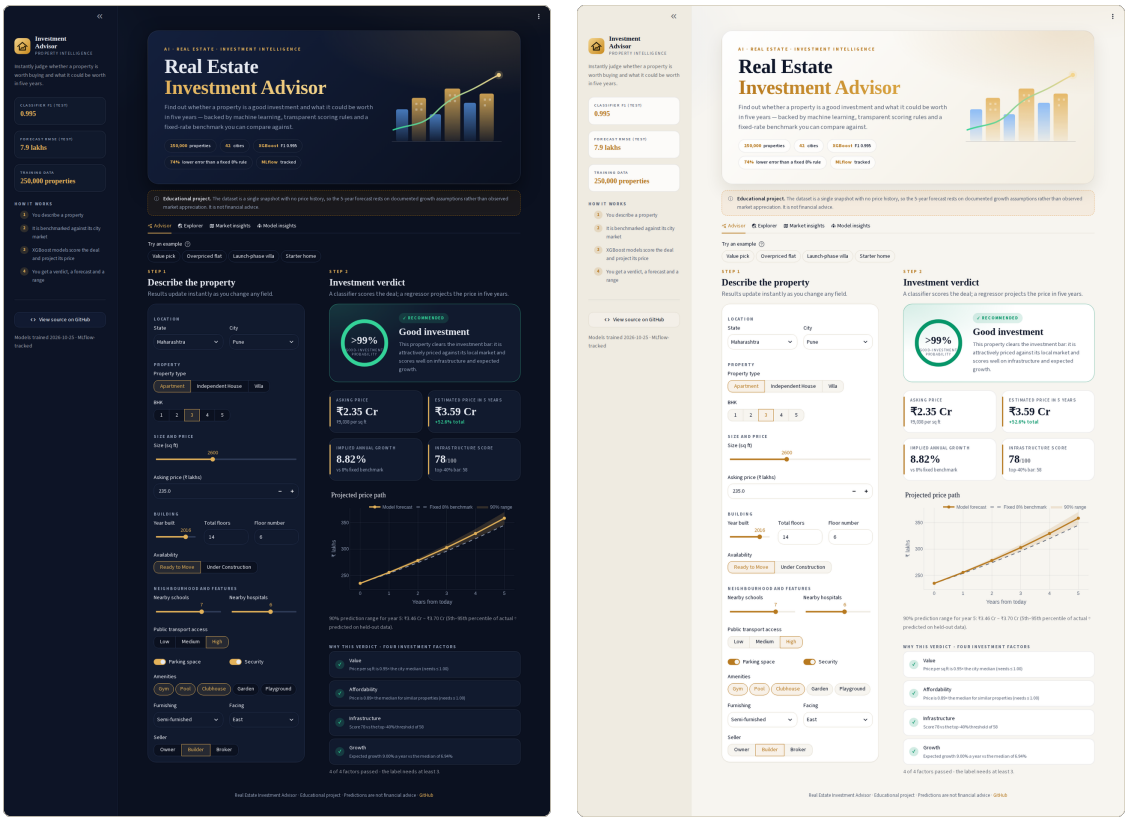


Figure 14. *The Advisor tab in dark (midnight and gold) and light (ivory and deep gold) themes. Streamlit follows the viewer's system setting and users can switch from the settings menu.*

- **Examples:** four one-click properties (Value pick, Overpriced flat, Launch-phase villa, Starter home) show how the verdict responds.
- **Live results:** the form re-runs only its own fragment, so every change updates the verdict in milliseconds.
- **Verdict ring:** probability that the property is a good investment; exact 0 or 1 are shown as `>99%` and `<1%`.
- **KPI cards:** asking price, five-year price, implied annual growth versus the 8% benchmark, and infrastructure score versus the top-40% bar.
- **Price path:** forecast with the 90% range and the fixed 8% benchmark. **Checklist:** which of the four factors passed and by how much.

Reading a negative verdict



Figure 15. A 'not recommended' result, with the factor-by-factor explanation.

A small, expensive, poorly connected flat in Mumbai is rejected. The verdict card turns red, the probability shows <1% and the checklist explains why: all four factors fail.

- Price per sq ft is **5.00×** the city median (the rule needs ≤ 1.00).
- The price is **1.64×** the median for similar properties.
- Infrastructure scores **12** against a bar of 58.
- Expected growth is **6.66%** against a median of 6.94%.

The forecast still shows a gain, because every property is assumed to appreciate; it falls below the fixed 8% benchmark line, which is a further warning sign.

Explorer tab

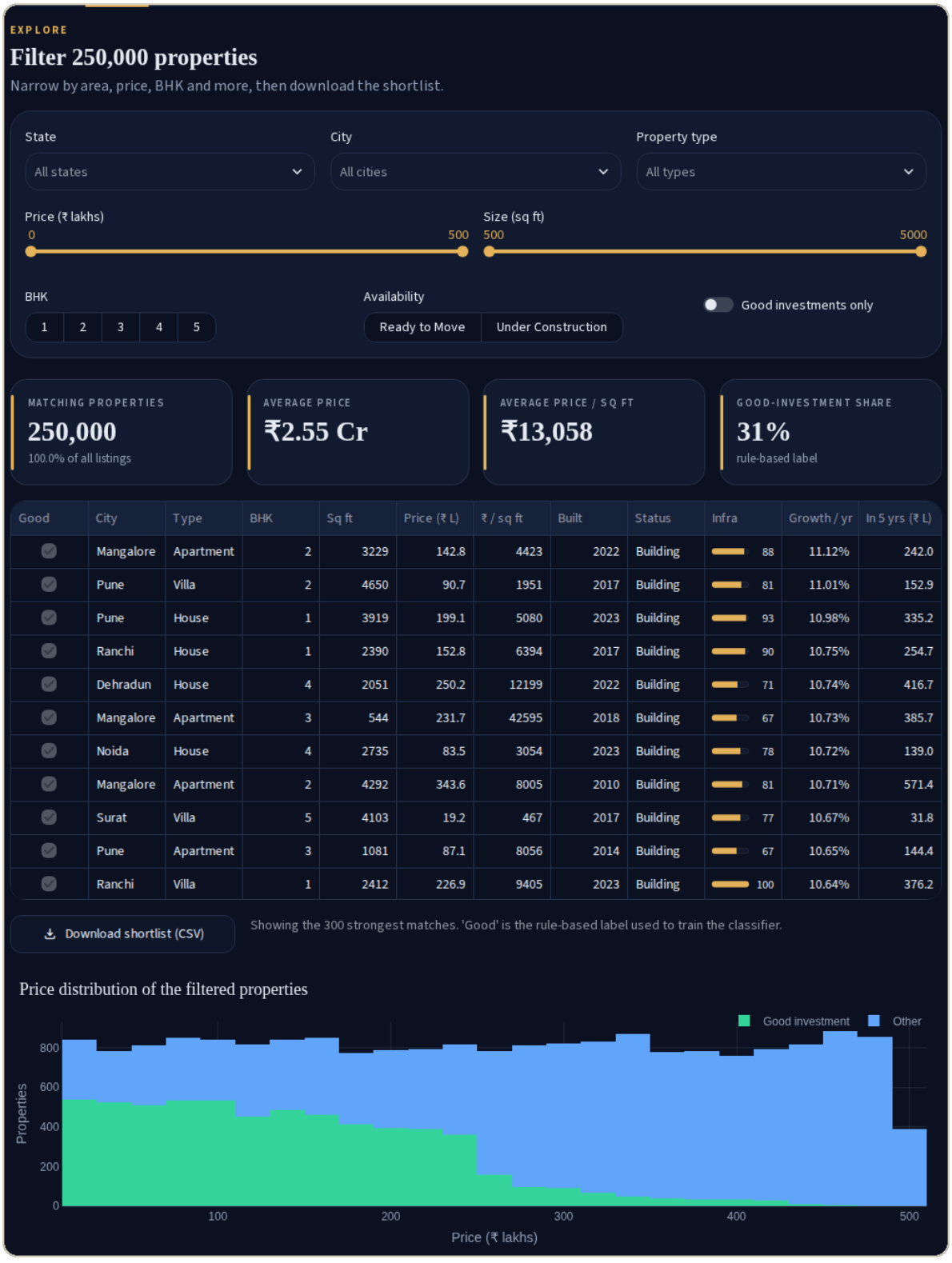


Figure 16. *The Explorer with filters, KPI cards, a table that sorts the strongest candidates first and a downloadable shortlist.*

Filters combine freely: choosing a state narrows the city list; the BHK and availability controls are multi-select; *Good investments only* keeps rows labelled good by the investment rules. The table shows the 300 strongest matches (good label first, then highest assumed growth) with progress bars for infrastructure and a CSV export of the raw values.

Market and model insights



Figure 17. *Left: Market insights (treemap, heatmap, rankings, trends). Right: Model insights (metrics, confusion matrix, benchmark, importance, honesty notes).*

Trend charts use **zero-based axes** on purpose: with a flat axis range the small differences in this dataset would look like dramatic swings. A callout reminds the reader that the data itself varies little.

Mobile layout

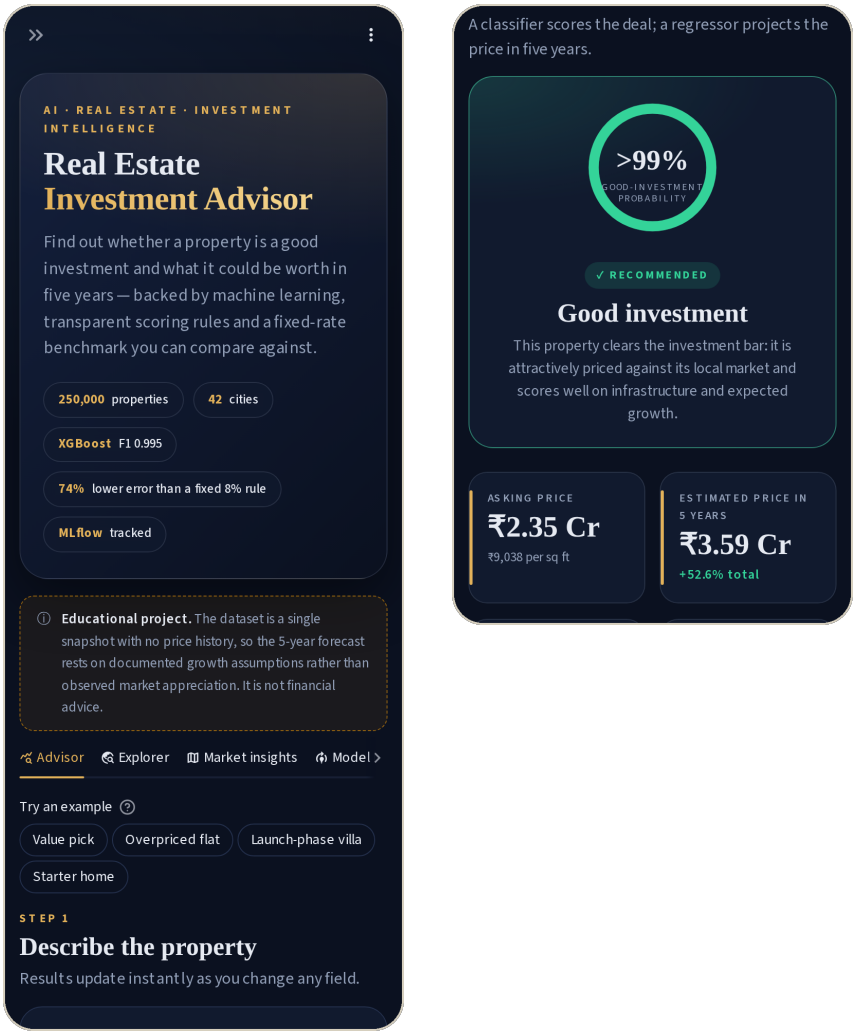
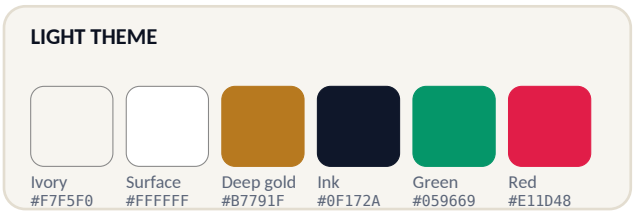


Figure 18. *Phone layout: the sidebar collapses, the hero art is hidden, the example pills wrap and form fields stack in one column.*

Design system





ELEMENT	CHOICE	REASON
Theming	Native `[theme.light]` and `[theme.dark]` palettes in config.toml, plus a design-system CSS layer	Every widget, table and chart follows the viewer's theme
Typography	Playfair Display for headlines, Inter for the interface	Editorial authority with high legibility
Components	KPI cards, verdict ring (SVG), factor checklist, callouts, pills and segmented controls	Scannable, consistent, accessible
Charts	Plotly with transparent backgrounds, themed fonts and gold / blue colour scales	Charts inherit the page theme
Performance	`st.fragment` per tab, cached data loaders, pre-aggregated tables	Widget changes do not re-run the whole app

Input validation and error handling

SCENARIO	BEHAVIOUR
Missing or empty field	`ValueError` listing the missing fields; shown as a clear message
Number out of range (e.g. size 5 sq ft)	`ValueError` such as "'Size_in_SqFt' must be between 100 and 20000"
Floor above the building height	Capped to the top floor; the form also clamps the field
City not seen in training	National benchmarks used; result flagged and an info notice shown
Price outside 10–500 lakhs or size outside 500–5,000 sq ft	Prediction runs, with an extrapolation warning
Model files missing	Friendly setup message with the command `python -m src.train`; the app stops safely
Unexpected exception	Caught, logged and shown as an error instead of a crash

SECTION 12

Code base and engineering practices

12

About 2,351 lines of commented Python, 13 automated tests and exact version pins.

Repository structure

```
Real-Estate-Investment-Advisor/
├── .streamlit/config.toml           # light and dark theme
├── app.py                           # Streamlit application
├── data/raw/india_housing_prices.csv
├── data/processed/                  # cleaned data + dashboard summaries
├── models/                          # pipelines, reference tables, metadata
├── reports/figures, reports/screenshots
├── src/
│   ├── config.py   data_loader.py   features.py   modeling.py
│   ├── train.py    predict.py       ui.py         logger.py
├── tests/test_pipeline.py, test_service.py
├── real-estate-investment-advisor.ipynb
├── docs/                            # this document
├── requirements.txt  requirements-dev.txt  .gitignore  README.md
```

Configuration

PARAMETER	VALUE	PURPOSE
RANDOM_STATE	42	Seed for every random operation
TEST_SIZE	0.2	Share of data held out
CURRENT_YEAR	2025	Reference year for property age
HORIZON_YEARS	5	Forecast horizon
GOOD_INVESTMENT_MIN_SCORE	3	Points (of 4) needed for the good label
BASELINE_GROWTH_RATE	0.08	Fixed-rate benchmark
GROWTH_BASE / CITY / INFRA	0.045 / 0.03 / 0.02	Growth formula weights
GROWTH_UNDER_CONSTRUCTION	0.008	Premium for under-construction projects
GROWTH_AGE_PENALTY	0.0005	Annual growth lost per year of age beyond 10
GROWTH_NOISE_STD	0.004	Market noise (reproducible)
GROWTH_MIN / GROWTH_MAX	0.03 / 0.12	Realistic growth limits

Error handling and logging

LAYER	MECHANISM
Data loading	`FileNotFoundError` with guidance; `DataValidationError` for empty, corrupt or incomplete files
Cleaning	Returns an audit report of every step; business-rule filters; imputation fallbacks
Modelling	Tuning failure falls back to defaults; save / load wrappers raise readable `OSError` messages
Training	Each model in its own try / except; MLflow failures are warnings; `RuntimeError` only if no model trains; non-zero exit code on failure
Prediction	`ValueError` for user mistakes, `RuntimeError` (logged) for unexpected faults
Application	Start-up checks, `st.stop()` with instructions, per-request try / except
Logging	One shared `get_logger()` with timestamp, level and module name

Automated tests

TEST	WHAT IT PROTECTS
`test_missing_file_raises`	A wrong path gives a clear FileNotFoundError
`test_schema_validation`	A CSV missing required columns is rejected
`test_cleaning_fixes_floors_and_adds_pps`	No floor above the building; rupee price per sq ft exists
`test_cleaning_removes_invalid_rows`	Non-positive prices and duplicates are dropped
`test_targets_are_valid`	Binary label; future price above current; growth within limits
`test_inference_features_match_training`	Single-row features equal training features
`test_outlier_report_shape`	IQR report has one row per column
`test_good_property_is_recommended`	Value pick passes all four factors
`test_overpriced_property_is_rejected`	Expensive, poorly connected flat is rejected
`test_missing_fields_raise_value_error`	Incomplete requests give a readable error
`test_out_of_range_value_raises_value_error`	Range validation works
`test_floor_above_building_is_capped`	Impossible floors never crash prediction
`test_unknown_city_uses_national_benchmarks`	Unseen city is handled and flagged

Reproducibility

LIBRARY	VERSION	LIBRARY	VERSION
Python	3.12 (3.11+)	streamlit	1.66.1
pandas	3.0.2	plotly	7.1.0
numpy	2.4.4	mlflow	3.17.2
scikit-learn	1.8.0	matplotlib	3.10.8
xgboost	3.4.1	seaborn	0.13.2
joblib	1.5.3	pytest	≥ 8.0

Models are pickled pipelines, so the runtime pins in `requirements.txt` are exact. A fixed seed, a stratified split and the audit trail from raw data to metrics make every number in this document reproducible.

SECTION 13

Deployment and operations

13

How to run, retrain and deploy the system, and what to do when something goes wrong.

Run locally

```
git clone https://github.com/13msrajput/Real-Estate-Investment-Advisor.git
cd Real-Estate-Investment-Advisor
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt                    # app + training + MLflow + notebook + tests

streamlit run app.py        # launch the application
pytest                      # run the 13 tests
```

Retrain the models

```
python -m src.train                 # full run: tuning + MLflow tracking + registry
python -m src.train --skip-tuning   # faster: default XGBoost parameters
python -m src.train --no-mlflow     # disable experiment tracking
```

Training rewrites the processed data, reference tables, model pipelines, metadata and figures. The app reads these artefacts at start-up, so restart it after retraining.

Deploy on Streamlit Community Cloud

1. Push the repository to GitHub (the data, models and theme are included, so no extra storage is needed).
2. Create a new app at share.streamlit.io, select the repository and set the main file to `app.py`.
3. Leave the requirements file as `requirements.txt` (runtime only, so the build is light and fast).
4. Deploy, then paste the public URL into the README and the notebook header.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Setup message: model files are missing	Artefacts not created	Run `python -m src.train`, then restart the app
Error loading a pickled model	Different scikit-learn / xgboost version	Install the exact versions in requirements.txt or retrain
Fonts look different offline	Google Fonts cannot load	System fonts are used; layout is unaffected
MLflow UI shows no runs	Store not created	Run training once; launch the UI from the project root

SECTION 14

Limitations, risks and roadmap

14

What the system cannot do, how the risks are managed, and where it goes next.

Limitations

- **No transaction dates.** Real appreciation cannot be learned; the five-year price is built from documented assumptions plus noise.
- **Prices are independent of size, BHK, city and type**, which is unrealistic; conclusions about real-world price drivers cannot be drawn.
- **City demand index** stretches a roughly 5% spread in median price per sq ft into a 0–1 scale, so city differences are modelling assumptions, not market evidence.
- **Rule-based label.** Classification accuracy mostly reflects rule recovery (section 9).
- **Locality** holds generic labels repeated across cities, so neighbourhood-level analysis is not possible.
- This is an educational project and **not financial advice**.

Risk register

RISK	IMPACT	MITIGATION
Users treat the forecast as advice	High	Prominent disclaimers, prediction ranges, extrapolation warnings, honesty callouts
Accuracy mistaken for real-world skill	High	Ablation and oracle benchmarks reported; rule-based nature stated on the Model insights tab
Library version drift breaks pickled models	Medium	Exact pins for runtime libraries; retrain command; clear error guidance
Out-of-range inputs	Medium	Validation limits and extrapolation warnings
Assumption changes alter both targets	Medium	All assumptions centralised in config; tests guard bounds
Fonts blocked in restricted networks	Low	System font fallbacks keep the layout intact

Roadmap

HORIZON	ITEM
Next	Replace simulated growth with real historical prices (for example RERA transactions or a city price index) and train a genuine time-series or panel model
Next	SHAP explanations per prediction; calibrated probabilities
Later	Geospatial features: distance to metro, IT parks and schools from open map data
Later	Serve the registered champions through an MLflow model server or API; CI/CD and Docker
Later	Drift monitoring and scheduled retraining with registry promotion

Maintenance checklist

- Re-train when new listings or real price history become available and compare runs in MLflow before moving the `champion` alias.
- Review the growth assumptions in `src/config.py` whenever the market view changes; they drive both targets.
- Re-run the tests and the notebook after any change to features or cleaning rules.
- Track data drift in the reference tables (city medians, thresholds); refit them with the models.

APPENDIX A

Data dictionary

All 23 raw columns with the values observed in the 250,000 listings.

COLUMN	TYPE	DESCRIPTION	OBSERVED
ID	int	Unique identifier for each property record	unique per row
State	text	State where the property is located	20 values
City	text	City of the property	42 values
Locality	text	Neighbourhood label (generic Locality_n)	500 values
Property_Type	text	Type of property	Apartment / Independent House / Villa
BHK	int	Number of bedrooms, hall and kitchen	1 – 5
Size_in_SqFt	int	Area in square feet	500 – 5000
Price_in_Lakhs	float	Price in lakhs (1 lakh = ₹100,000)	10 – 500
Price_per_SqFt	float	Price ÷ area, in lakhs, rounded to 2 decimals	0 – 0.99
Year_Built	int	Year of construction	1990 – 2023
Furnished_Status	text	Furnishing level	Furnished / Semi-furnished / Unfurnished
Floor_No	int	Floor of the property	0 – 30
Total_Floors	int	Floors in the building	1 – 30
Age_of_Property	int	2025 − Year_Built	2 – 35
Nearby_Schools	int	Number of nearby schools	1 – 10
Nearby_Hospitals	int	Number of nearby hospitals	1 – 10
Public_Transport_Accessibility	text	Access to buses, metro, train	High / Low / Medium
Parking_Space	text	Parking available	No / Yes
Security	text	Security provided	No / Yes
Amenities	text	In-society amenities	subset of Gym, Pool, Clubhouse, Garden, Playground
Facing	text	Direction the property faces	East / North / South / West
Owner_Type	text	Who is selling	Broker / Builder / Owner
Availability_Status	text	Current status	Ready_to_Move / Under_Construction

APPENDIX B

Model feature dictionary

B

The 30 inputs of both models, in the order used by the pipelines.

#	FEATURE	KIND	DEFINITION
1	`BHK`	numeric	Bedrooms, hall, kitchen
2	`Size_in_SqFt`	numeric	Area in sq ft
3	`Price_in_Lakhs`	numeric	Asking price
4	`Price_per_SqFt_Capped`	numeric	Price per sq ft, winsorised at 1st/99th percentile
5	`Age_of_Property`	numeric	2025 − Year_Built
6	`Total_Floors`	numeric	Floors in the building
7	`Floor_Ratio`	numeric	Floor_No ÷ Total_Floors
8	`Size_per_BHK`	numeric	Size ÷ BHK
9	`Nearby_Schools`	numeric	Schools nearby
10	`Nearby_Hospitals`	numeric	Hospitals nearby
11	`School_Density_Score`	numeric	Schools ÷ city average
12	`Transport_Score`	numeric	Low 1, Medium 2, High 3
13	`Parking_Flag`	numeric	1 if parking
14	`Security_Flag`	numeric	1 if security
15	`Amenity_Count`	numeric	Number of amenities (0–5)
16	`Has_Gym`	numeric	Amenity flag
17	`Has_Pool`	numeric	Amenity flag
18	`Has_Clubhouse`	numeric	Amenity flag
19	`Has_Garden`	numeric	Amenity flag
20	`Has_Playground`	numeric	Amenity flag
21	`Infra_Score`	numeric	Weighted 0–100 infrastructure score
22	`PPS_vs_City_Median`	numeric	Price per sq ft ÷ city median
23	`Price_vs_City_Median`	numeric	Price ÷ median for city and type
24	`Is_Under_Construction`	numeric	1 if under construction
25	`City_Demand_Index`	numeric	Rank of city median price per sq ft, 0–1
26	`City`	categorical	One-hot encoded (42 cities)
27	`Property_Type`	categorical	One-hot encoded (3)
28	`Furnished_Status`	categorical	One-hot encoded (3)
29	`Facing`	categorical	One-hot encoded (4)
30	`Owner_Type`	categorical	One-hot encoded (3)

Excluded on purpose: `Locality` (generic labels), `State` (implied by City), `Expected_Growth_Rate` (a direct function of the other inputs that would leak the target) and the other label-defining columns.

APPENDIX C

Prediction service reference

How to call the same service the application uses.

```
from src.predict import AdvisorService

service = AdvisorService()          # loads pipelines, reference tables, metadata
result = service.predict({
    "City": "Pune", "Property_Type": "Apartment", "BHK": 3, "Size_in_SqFt": 2600,
    "Price_in_Lakhs": 235, "Year_Built": 2016, "Furnished_Status": "Semi-furnished",
    "Floor_No": 6, "Total_Floors": 14, "Nearby_Schools": 7, "Nearby_Hospitals": 6,
    "Public_Transport_Accessibility": "High", "Parking_Space": "Yes", "Security": "Yes",
    "Amenities": ["Gym", "Pool", "Clubhouse"], "Facing": "East", "Owner_Type": "Builder",
    "Availability_Status": "Ready_to_Move"})
```

Input fields and limits

FIELD	ALLOWED VALUES / RANGE
`City`	Any text; unseen cities use national benchmarks and are flagged
`Property_Type`	Apartment, Independent House, Villa
`BHK`	1 – 10
`Size_in_SqFt`	100 – 20,000
`Price_in_Lakhs`	1 – 10,000
`Year_Built`	1900 – 2030
`Floor_No / Total_Floors`	0 – 200 / 1 – 200 (floor capped at Total_Floors)
`Nearby_Schools / Nearby_Hospitals`	0 – 10
`Public_Transport_Accessibility`	Low, Medium, High
`Parking_Space / Security`	Yes, No
`Amenities`	List or comma-separated text of Gym, Pool, Clubhouse, Garden, Playground
`Furnished_Status / Facing / Owner_Type / Availability_Status`	Furnished, Semi-furnished, Unfurnished / North, South, East, West / Owner, Builder, Broker / Ready_to_Move, Under_Construction

Returned fields

KEY	MEANING
`good_investment, probability`	Verdict and probability of the 'good' class
`current_price, future_price`	Asking price and five-year forecast (₹ lakhs)
`future_price_low, future_price_high`	90% prediction range for year five
`appreciation_pct, implied_cagr_pct`	Total gain and implied annual growth
`expected_growth_pct`	Noise-free growth used by the label rules
`infra_score`	0 – 100 infrastructure score
`price_vs_city_median, pps_vs_city_median`	Ratios used by the value and affordability factors
`price_per_sqft`	Price per sq ft in rupees
`factors`	Dictionary of the four factor results (booleans)
`city_known`	False when national benchmarks replaced a city benchmark

APPENDIX D

Glossary

D

Terms used in this document.

TERM	MEANING
Lakh / crore	Indian units: 1 lakh = 100,000; 100 lakhs = 1 crore (10 million).
BHK	Bedrooms, hall and kitchen: the standard size label for Indian homes.
Winsorisation	Capping extreme values at chosen percentiles instead of deleting them.
IQR rule	Flags values beyond 1.5 × the inter-quartile range from the quartiles.
F1 score	Harmonic mean of precision and recall.
ROC-AUC	Probability that a random positive is ranked above a random negative.
RMSE / MAE / MAPE	Root mean squared / mean absolute / mean absolute percentage error.
Ablation	Removing a group of features to measure how much the model depends on them.
Oracle	A hypothetical model that knows the exact data-generating formula; it shows the error floor.
Rule recovery	A model learning the rules that define its own label rather than an independent signal.
Prediction interval	A range expected to contain the true value with a stated probability (here 90%).
Fragment	A Streamlit block that re-runs on its own when its widgets change.
Champion	The registered model version currently promoted for use (MLflow alias).

APPENDIX E

About the author

E

Contact details and project links.

Mohit Singh Rajput is an AI/ML engineer. This project was designed, implemented, tested and documented end to end as a portfolio-grade machine-learning application.

CHANNEL	LINK
GitHub repository	github.com/13msrajput/Real-Estate-Investment-Advisor
Live app	real-estate-investment-advisor-app.streamlit.app
GitHub profile	github.com/13msrajput
Portfolio	13msrajput.github.io
LinkedIn	linkedin.com/in/13msrajput
Kaggle	kaggle.com/13msrajput
LeetCode	leetcode.com/u/13msrajput
Email	mohitsinghrajput1307@gmail.com

THANK YOU

If this project was useful, a star on the GitHub repository is appreciated. Predictions are for education only and are not financial advice.